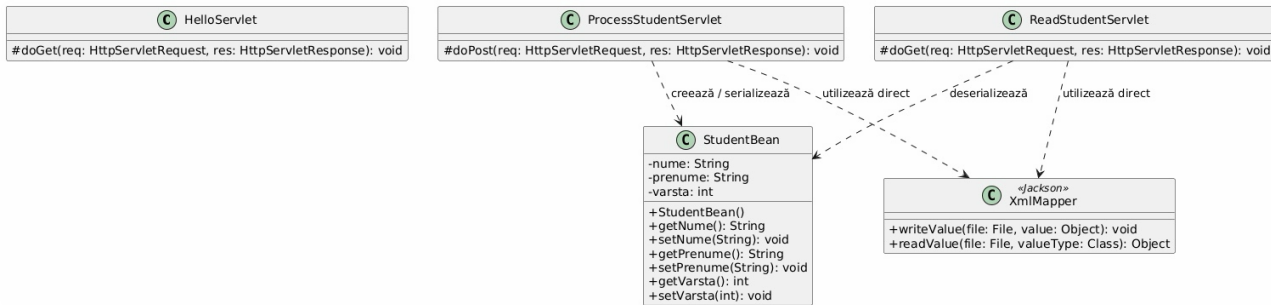
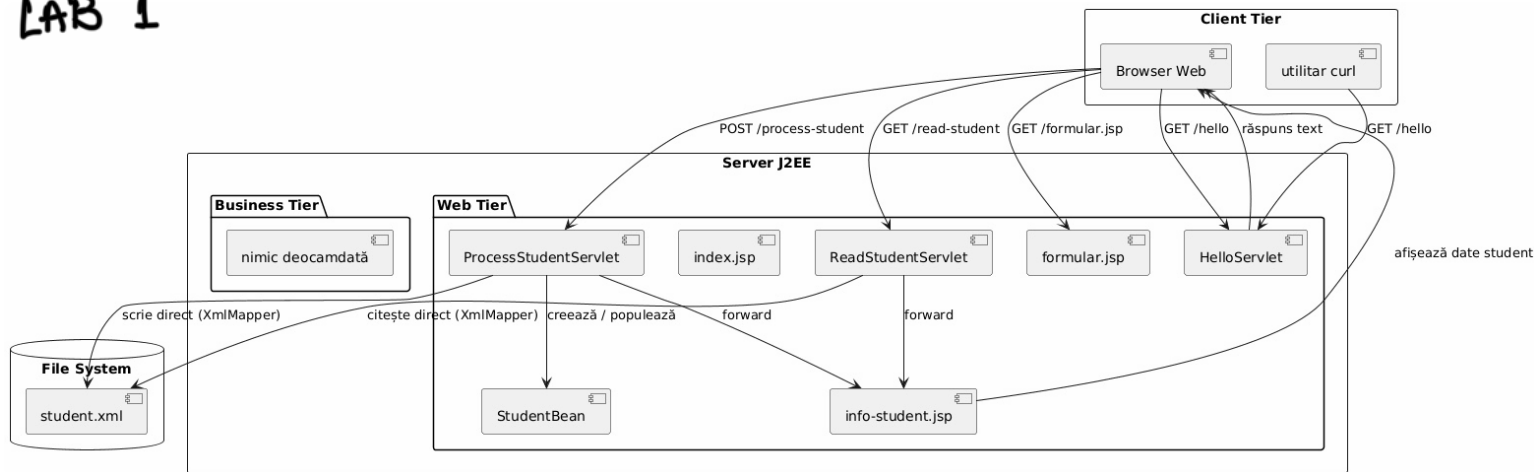


LAB 1



Aceasta este diagrama de microservicii și clase pentru exemplul din lab.

De ce nu sunt respectate principiile SOLID?

S - Single Responsibility

ProcessStudentServlet → face trei lucruri în același loc: → HTTP handling
→ business logic (calcul cu mapster)
→ persistență (serie XML)

O - Open/Closed

Dacă vrem să schimbăm formatul de salvare din XML în JSON sau SQLite nu putem. În cod este folosit XmlMapper. Trebuie înlocuit cu ObjectMapper și folosite clase separate în funcție de format

L - Linked Substitution

Nu există ierarhie de clase pentru persistență. ProcessStudentServlet are explicit că lucrează cu XmlMapper - nu poate fi înlocuit cu nimic.

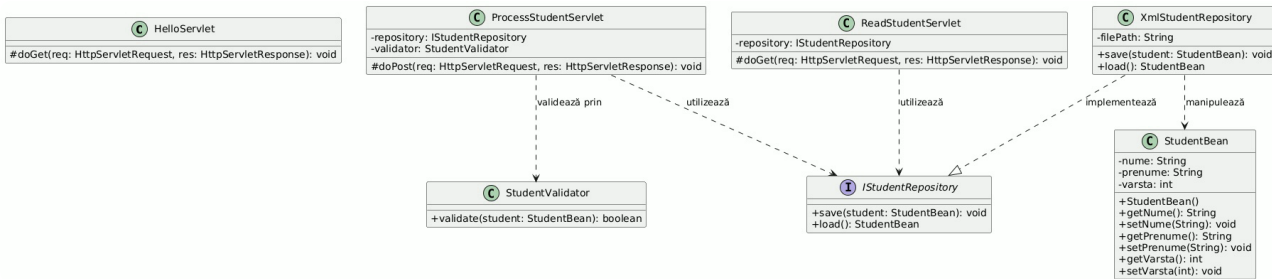
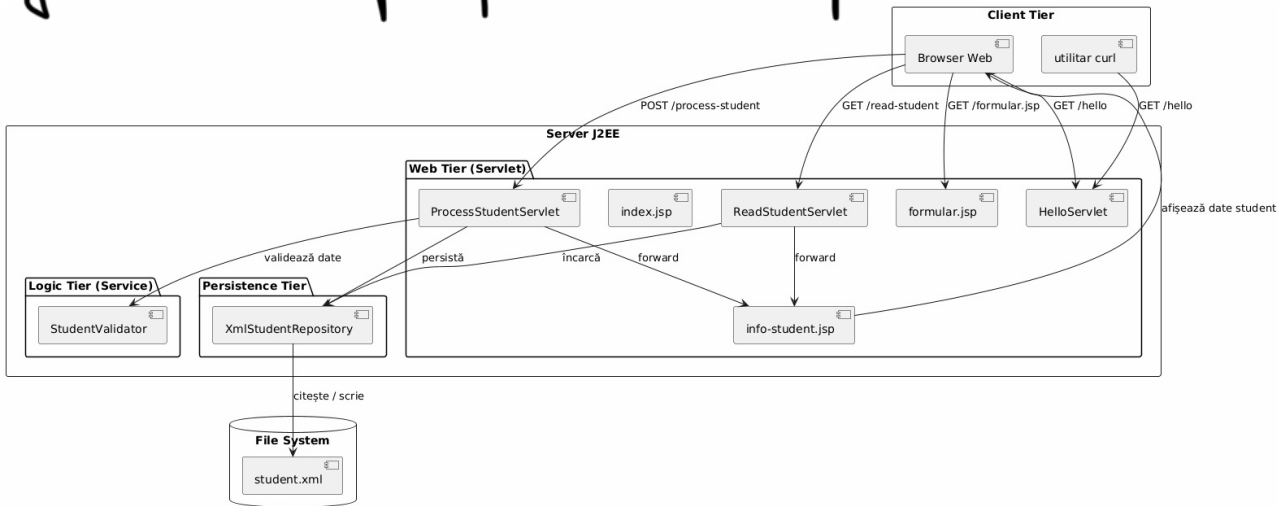
J- Interface Segregation

Nu există interfețe. Servuletele depind direct de clase concrete.

D- Dependency Inversion

Servitul depinde direct de Xml Mapper.

Diagrammele ce principiile SOLID respectate:



De ce sunt respectate ?

5 ProcessStudent Servlet se ocupă acum doar cu HTTP
Student Validator - cu logica
Xml Student Repository - cu persistența.

0 Pentru a schimba formatul de salvare doar adăugăm o clasă.

L Am adăugat interfața IStudentRepository.

```
IStudentRepository repo = new XmlStudentRepository();
```

```
// sau
```

```
IStudentRepository repo = new JsonStudentRepository();
```

```
// servlet-ul funcționează identic în ambele cazuri
```

J `JStudentRepository` are exact două metode: `save()` și `load()`.
Nici o clasă nu e forțată să implementeze metode de care nu are
nevoie.

D În `ProcessStudentServlet` avem obiectul `repository` de tip
`JStudentRepository` care aduce abstracție după tip de salvare